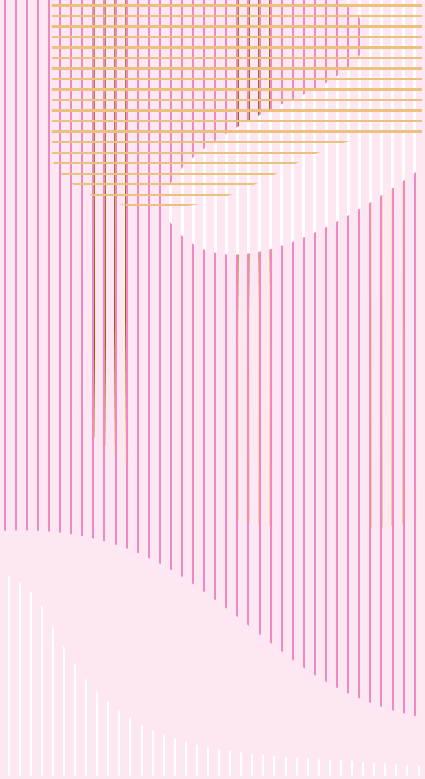


1. Phát biểu nào sau đây **sai** về các thành phần của một tiến trình trong hệ điều hành?

   a) Text section chứa mã lệnh của chương trình.
   b) Stack section dùng để lưu tham số hàm, biến cục bộ và địa chỉ trả về.
   c) Heap section chứa các biến toàn cục của chương trình.
   d) Data section chứa các biến toàn cục và biến static.

   Đáp án: C - Heap section không chứa biến toàn cục. Heap được dùng cho vùng nhớ cấp phát động trong thời gian chạy, còn biến toàn cục và biến static được lưu trong Data section.

2. Cho đoạn chương trình C++ sau:

```
int main() {
    int *a = new int(2);
}
```

   Phát biểu nào sau đây là đúng?

   a) Biến con trỏ `a` nằm trong Heap, còn giá trị 2 nằm trong Stack.
   b) Biến con trỏ `a` nằm trong Stack, còn vùng nhớ chứa giá trị 2 nằm trong Heap.
   c) Cả biến `a` và giá trị 2 đều nằm trong Stack.
   d) Cả biến `a` và giá trị 2 đều nằm trong Heap.

   Đáp án: B - Biến con trỏ `a` là biến cục bộ nên được lưu trong Stack. Toán tử `new` cấp phát động một vùng nhớ trong Heap để chứa giá trị 2.

3. Phát biểu nào sau đây đúng về khái niệm *passive* và *active* trong hệ điều hành?

   a) Process là thực thể thụ động vì chỉ lưu trên ổ đĩa.
   b) Program là thực thể hoạt động vì luôn được CPU thực thi.
   c) Program là thực thể thụ động, còn Process là thực thể hoạt động.
   d) Program và Process đều là thực thể hoạt động.

   Đáp án: C - Program chỉ là tập tin chương trình được lưu trữ trên ổ đĩa nên là thực thể thụ động (passive). Khi chương trình được nạp vào bộ nhớ và thực thi, nó trở thành Process — một thực thể hoạt động (active) có trạng thái, thanh ghi CPU và tài nguyên riêng.

4. Trong sơ đồ trạng thái tiến trình của hệ điều hành, trường hợp nào sau đây làm tiến trình chuyển từ trạng thái *Running* sang *Ready*?

   a) Tiến trình yêu cầu thao tác nhập/xuất (I/O).
   b) Bộ định thời hết quantum và CPU bị thu hồi.
   c) Tiến trình hoàn thành thực thi.
   d) Tiến trình vừa được tạo.

   Đáp án: B - Khi hết thời gian quantum trong các giải thuật định thời ưu tiên ngắt hoặc Round Robin, hệ điều hành sẽ thu hồi CPU và chuyển tiến trình từ Running về Ready để chờ được cấp CPU lại.

5. Trong mô hình trạng thái tiến trình, khi nào một tiến trình chuyển từ trạng thái *Waiting* sang *Ready*?

   a) Khi tiến trình được cấp CPU để thực thi.
   b) Khi tiến trình yêu cầu thao tác nhập/xuất (I/O).
   c) Khi sự kiện đang chờ hoàn thành, chẳng hạn thao tác I/O kết thúc.
   d) Khi tiến trình bị hệ điều hành thu hồi CPU.

   Đáp án: C - Tiến trình ở trạng thái Waiting đang chờ một sự kiện xảy ra (ví dụ hoàn thành I/O). Khi sự kiện đó kết thúc, tiến trình sẽ được đưa sang trạng thái Ready để chờ cấp CPU.

6. Trường hợp nào làm tiến trình chuyển từ trạng thái *Running* sang *Waiting*?

   a) Tiến trình hoàn thành thực thi.
   b) Tiến trình yêu cầu thao tác nhập/xuất (I/O).
   c) Bộ định thời hết quantum.
   d) Tiến trình vừa được tạo.

Đáp án: B - Khi tiến trình đang chạy cần chờ thao tác I/O hoặc một sự kiện nào đó hoàn thành, nó sẽ chuyển sang trạng thái Waiting.

7. Trường hợp nào làm tiến trình chuyển từ trạng thái *Running* sang *Terminated*?

a) Tiến trình hoàn thành việc thực thi.
b) Tiến trình yêu cầu thao tác I/O.
c) Bộ định thời hết quantum.
d) Có tiến trình ưu tiên cao hơn xuất hiện.

Đáp án: A - Khi tiến trình thực hiện xong toàn bộ chương trình và kết thúc thực thi, hệ điều hành sẽ chuyển nó sang trạng thái Terminated.

8. Thành phần nào sau đây **không** được lưu trong PCB (*Process Control Block*) của tiến trình?

a) Program Counter.
b) Thông tin thanh ghi CPU.
c) Trạng thái tiến trình.
d) Mã lệnh của toàn bộ chương trình.

Đáp án: D - PCB chỉ lưu thông tin quản lý tiến trình như trạng thái, thanh ghi CPU, program counter, thông tin định thời và quản lý bộ nhớ. Mã lệnh chương trình được lưu ở Text section chứ không nằm trong PCB.

9. Mục đích chính của *Process Scheduling* trong hệ điều hành là gì?

a) Tăng kích thước bộ nhớ chính của hệ thống.
b) Quyết định tiến trình nào được cấp CPU để thực thi.
c) Lưu trữ mã lệnh của chương trình vào ổ đĩa.
d) Chuyển đổi chương trình nguồn thành mã máy.

Đáp án: B - Process Scheduling có nhiệm vụ lựa chọn tiến trình tiếp theo được cấp CPU nhằm sử dụng tài nguyên hiệu quả và tối ưu hiệu năng hệ thống.

10. Phát biểu nào sau đây đúng khi so sánh *Ready Queue* và *Wait Queue* trong hệ điều hành?

a) Ready Queue chứa các tiến trình đang chờ hoàn thành I/O.
b) Wait Queue chứa các tiến trình đang sẵn sàng nhận CPU.
c) Ready Queue chứa các tiến trình chờ được cấp CPU, còn Wait Queue chứa các tiến trình đang chờ sự kiện hoặc I/O.
d) Cả Ready Queue và Wait Queue đều chứa các tiến trình đang thực thi.

Đáp án: C - Ready Queue lưu các tiến trình đã sẵn sàng chạy và chỉ chờ CPU, trong khi Wait Queue lưu các tiến trình đang chờ một sự kiện xảy ra như hoàn thành thao tác I/O.

11. Trong thực tế, hệ điều hành cần thực hiện *Context Switch* trong trường hợp nào sau đây?

a) Khi tiến trình đang chạy yêu cầu thao tác I/O hoặc bị hết quantum CPU.
b) Khi người dùng mở thêm một thư mục trong ổ đĩa.
c) Khi bộ nhớ RAM còn trống nhiều.
d) Khi chương trình được biên dịch sang mã máy.

Đáp án: A - Context Switch xảy ra khi CPU phải chuyển từ tiến trình này sang tiến trình khác, thường do tiến trình đang chạy bị chặn bởi I/O, hết quantum hoặc bị tiến trình ưu tiên cao hơn giành CPU.

12. Cho đoạn chương trình UNIX/Linux sau:

```
int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        return 0;
    }
    else {
        sleep(10);
```

```
    }
}
```

Điều gì sẽ xảy ra sau khi tiến trình con kết thúc?

a) Tiến trình con trở thành orphan process.
b) Tiến trình con trở thành zombie process.
c) Tiến trình cha bị kết thúc ngay lập tức.
d) Tiến trình con tự động gọi `exec()`.

Đáp án: B - Tiến trình con kết thúc nhưng tiến trình cha không gọi `wait()` để thu hồi trạng thái kết thúc của con. Vì vậy, tiến trình con sẽ ở trạng thái zombie cho đến khi cha kết thúc hoặc gọi `wait()`.

13. Xét đoạn chương trình C/C++ sau:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    fork();

    if (fork() == 0) {
        fork();
        printf("A ");
    } else {
        fork();
        fork();
        printf("B ");
    }

    return 0;
}
```

Hỏi chương trình in ra tổng cộng bao nhiêu lần ký tự `"A "` và `"B "`?

a) 4 lần A và 8 lần B
b) 1 lần A và 3 lần B
c) 2 lần A và 3 lần B
d) 3 lần A và 3 lần B

*Đáp án: C - fork sẽ tạo ra 2 và fork if sẽ tạo ra 4 mỗi if 1 cái và bên trong sẽ 2*2 A và 2*2*2 B*

14. Trong UNIX/Linux, ý nghĩa chính của hàm `exec()` là gì?

a) Tạo ra một tiến trình con mới giống tiến trình cha.
b) Tạm dừng tiến trình hiện tại để chờ tiến trình khác.
c) Thay thế không gian bộ nhớ của tiến trình hiện tại bằng một chương trình mới.
d) Hủy tiến trình con và giải phóng toàn bộ tài nguyên.

Đáp án: C - Hàm `exec()` không tạo tiến trình mới mà thay thế mã lệnh, dữ liệu và không gian bộ nhớ của tiến trình hiện tại bằng một chương trình khác để tiếp tục thực thi.

15. Sau khi gọi hàm `fork()` trong UNIX/Linux, tiến trình con có đặc điểm nào giống tiến trình cha?

a) Có cùng PID với tiến trình cha.
b) Chia sẻ cùng Program Counter đang chạy.
c) Có bản sao gần như giống hệt không gian địa chỉ và mã lệnh của tiến trình cha.
d) Tự động thực thi một chương trình mới bằng `exec()`.

Đáp án: C - Sau khi `fork()`, tiến trình con được tạo như một bản sao của tiến trình cha, bao gồm mã lệnh, dữ liệu và trạng thái thực thi. Tuy nhiên, tiến trình con có PID riêng và thực thi độc lập với tiến trình cha.

16. Xét đoạn chương trình C trong UNIX/Linux:

```
int x = 5;
```

```
int main() {
    pid_t pid = fork();

    if (pid == 0) {
        x = 10;
    } else {
        x = 20;
        wait(NULL);
    }
}
```

Phát biểu nào sau đây đúng?

a) Hai tiến trình cha và con cùng chia sẻ biến x, nên giá trị cuối cùng của x là 20.

b) Sau `fork()`, tiến trình cha và con có không gian bộ nhớ riêng, nên thay đổi x không ảnh hưởng lẫn nhau.

c) Tiến trình con ghi đè trực tiếp lên biến x của tiến trình cha.

d) Giá trị x luôn được đồng bộ giữa hai tiến trình trong thời gian chạy.

Đáp án: B - Sau khi `fork()`, tiến trình con và tiến trình cha có không gian địa chỉ riêng (copy-on-write). Do đó, khi một tiến trình thay đổi biến `x`, sự thay đổi này không ảnh hưởng đến tiến trình còn lại.

17. Phát biểu nào sau đây đúng về cơ chế *Shared Memory* trong giao tiếp giữa các tiến trình?

a) Hệ điều hành tự động đồng bộ hóa truy cập vào vùng nhớ chia sẻ.

b) Các tiến trình giao tiếp qua Shared Memory và tự quản lý việc đồng bộ hóa.

c) Shared Memory chỉ dùng để truyền thông điệp giữa các tiến trình qua kernel.

d) Shared Memory không thể dùng trong bài toán Producer - Consumer.

Đáp án: B - Với Shared Memory, các tiến trình tự truy cập vào vùng nhớ chung và tự chịu trách nhiệm đồng bộ hóa. Hệ điều hành không tự động đảm bảo tránh xung đột dữ liệu.

18. Vấn đề *Producer-Consumer* trong hệ điều hành liên quan chủ yếu đến điều gì?

a) Quản lý CPU scheduling giữa nhiều tiến trình.

b) Đồng bộ hóa truy cập vào bộ nhớ dùng chung giữa các tiến trình.

c) Tối ưu hóa dung lượng ổ đĩa cứng.

d) Chuyển đổi giữa user mode và kernel mode.

Đáp án: B - Producer-Consumer là bài toán điển hình của đồng bộ hóa, trong đó các tiến trình Producer và Consumer cùng truy cập một buffer dùng chung, cần cơ chế synchronization để tránh race condition, tràn buffer hoặc đọc dữ liệu không hợp lệ.

19. Phương pháp nào sau đây thường được sử dụng để giải quyết bài toán *Producer-Consumer*?

a) Sử dụng chỉ CPU scheduling như FCFS hoặc Round Robin.

b) Sử dụng các cơ chế đồng bộ hóa như semaphore hoặc mutex.

c) Tăng kích thước bộ nhớ RAM để tránh xung đột.

d) Cho phép Producer và Consumer chạy trên các CPU khác nhau mà không cần đồng bộ.

Đáp án: B - Bài toán Producer-Consumer được giải quyết bằng các cơ chế đồng bộ hóa như semaphore, mutex hoặc monitor để đảm bảo không xảy ra race condition khi truy cập buffer dùng chung.

20. Trong cơ chế *Message Passing*, có bao nhiêu cách giao tiếp giữa các tiến trình?

a) 1 cách: chỉ giao tiếp trực tiếp giữa các tiến trình.

b) 2 cách: giao tiếp trực tiếp và giao tiếp gián tiếp.

c) 3 cách: trực tiếp, gián tiếp và qua shared memory.

d) 4 cách: trực tiếp, gián tiếp, shared memory và pipe.

Đáp án: B - Message Passing có 2 hình thức chính: giao tiếp trực tiếp (Direct) và giao tiếp gián tiếp (Indirect thông qua mailbox/port).

21. Nhược điểm của cơ chế *Direct Communication* (giao tiếp trực tiếp) so với *Indirect Communication* là gì?

a) Linh hoạt hơn vì không cần biết tên tiến trình.
b) Dễ mở rộng trong hệ thống lớn có nhiều tiến trình.
c) Các tiến trình phải biết rõ tên của nhau nên kém linh hoạt và khó mở rộng.
d) Không cần bất kỳ cơ chế đồng bộ nào.

Đáp án: C - Direct Communication yêu cầu tiến trình gửi và nhận phải biết rõ tên nhau, nên kém linh hoạt và khó mở rộng hơn so với Indirect Communication (dùng mailbox/port).

22. Phát biểu nào sau đây đúng về *Blocking* và *Non-blocking* trong cơ chế truyền thông điệp (Message Passing)?

a) Blocking send/receive luôn không làm tiến trình dừng lại.
b) Non-blocking send/receive khiến tiến trình phải chờ cho đến khi thao tác hoàn tất.
c) Blocking khiến tiến trình phải chờ cho đến khi gửi/nhận xong, còn Non-blocking cho phép tiếp tục thực thi ngay.
d) Blocking và Non-blocking chỉ khác nhau ở tốc độ CPU.

Đáp án: C - Blocking communication khiến tiến trình phải chờ đến khi gửi/nhận hoàn tất, trong khi Non-blocking cho phép tiến trình tiếp tục thực thi mà không bị chặn.

23. Lợi ích nào sau đây **không phải** là ưu điểm của đa luồng (multithreading)?

a) Tăng tính phản hồi (Responsiveness) của chương trình.
b) Giảm chi phí tạo và chuyển đổi ngữ cảnh so với tiến trình.
c) Các luồng có không gian địa chỉ riêng hoàn toàn độc lập như các tiến trình.
d) Tận dụng tốt kiến trúc đa lõi (multicore) để tăng hiệu năng.

Đáp án: C - Trong đa luồng, các luồng trong cùng một tiến trình chia sẻ chung không gian địa chỉ và tài nguyên, không phải độc lập hoàn toàn như các tiến trình.

24. Phát biểu nào sau đây đúng khi so sánh giữa *Process* và *Thread*?

a) Process nhẹ hơn Thread vì không cần quản lý bộ nhớ.
b) Thread có tính độc lập hoàn toàn, không chia sẻ tài nguyên với các Thread khác.
c) Process có không gian bộ nhớ riêng, trong khi các Thread trong cùng Process chia sẻ chung bộ nhớ.
d) Khi một Thread bị chặn thì toàn bộ hệ thống sẽ dừng hoạt động.

Đáp án: C - Process là đơn vị thực thi độc lập với không gian địa chỉ riêng, còn các Thread trong cùng một Process chia sẻ chung bộ nhớ và tài nguyên, giúp giao tiếp nhanh hơn nhưng cần đồng bộ hóa.

25. Chọn phát biểu đúng về sự khác biệt giữa process creation và thread creation:

a) Process creation nhẹ hơn thread creation vì không cần cấp phát bộ nhớ
b) Thread creation nặng hơn process creation vì phải tạo không gian địa chỉ riêng
c) Process creation là heavy-weight do cần cấp phát nhiều tài nguyên, thread creation là light-weight vì chia sẻ tài nguyên trong cùng process
d) Cả process và thread creation đều có chi phí như nhau trong mọi hệ điều hành

Đáp án: C

26. Trong cùng một process, các thread thường chia sẻ thành phần nào sau đây?

a) Program Counter, Stack và Register
b) Stack, Register và Thread ID
c) Code section, Data section và vùng nhớ heap
d) Program Counter, Stack và Heap

Đáp án: C

27. Phát biểu nào sau đây đúng khi so sánh *Concurrency* và *Parallelism*?

a) Concurrency chỉ xảy ra trên hệ thống đa lõi, còn Parallelism chỉ trên đơn lõi.
b) Parallelism là khả năng thực hiện nhiều tác vụ đúng cùng một thời điểm, còn Concurrency là nhiều tác vụ có tiến triển xen kẽ nhau.
c) Concurrency và Parallelism là hai khái niệm giống hệt nhau.
d) Parallelism không cần phần cứng hỗ trợ CPU đa lõi.

Đáp án: B - Parallelism là thực hiện nhiều tác vụ cùng thời điểm (thường trên nhiều lõi), còn Concurrency là xử lý xen kẽ nhiều tác vụ, tạo cảm giác đồng thời, thường trên một lõi nhờ lập lịch CPU.

28. Trong thực tế hệ điều hành hiện đại (ví dụ Windows, Linux), cơ chế nào sau đây thường được sử dụng để vừa tăng hiệu năng vừa tận dụng đa lõi CPU?

a) Chỉ sử dụng Concurrency trên một CPU đơn lõi.
b) Chỉ sử dụng Parallelism mà không cần lập lịch CPU.
c) Kết hợp cả Concurrency và Parallelism.
d) Không sử dụng cả Concurrency và Parallelism.

Đáp án: C - Hệ điều hành hiện đại kết hợp cả Concurrency (quản lý nhiều tác vụ xen kẽ qua scheduling) và Parallelism (thực thi song song trên nhiều lõi CPU) để tối ưu hiệu năng và khả năng đáp ứng hệ thống.

29. Áp dụng Định luật Amdahl, một chương trình có 30% phần công việc phải thực hiện tuần tự, phần còn lại có thể song song hóa hoàn toàn. Hệ thống sử dụng 4 lõi xử lý.

Mức tăng tốc tối đa (speedup) gần đúng là bao nhiêu?

a) 1.54 b) 2.00 c) 2.50 d) 2.105

Đáp án: D - Theo Định luật Amdahl:

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

Với $S = 0.3$, $N = 4$:

$$speedup = \frac{1}{0.3 + \frac{0.7}{4}} = \frac{1}{0.3 + 0.175} = \frac{1}{0.475} \approx 2.105$$

30. Chọn phát biểu đúng về asynchronous threading và synchronous threading:

a) Asynchronous threading yêu cầu thread cha phải chờ thread con hoàn thành
b) Synchronous threading cho phép thread cha tiếp tục chạy mà không cần chờ thread con
c) Asynchronous threading cho phép thread cha không chờ thread con, còn synchronous threading yêu cầu phải chờ
d) Cả hai đều không có liên quan đến việc chờ hay không chờ giữa các thread

Đáp án: C - Asynchronous threading không chờ thread con, còn synchronous threading thì có chờ (thường thông qua join).

31. Một hệ thống sử dụng mô hình đa luồng, trong đó mỗi luồng người dùng được ánh xạ trực tiếp vào một luồng hạt nhân riêng biệt. Mô hình này giúp tận dụng tốt CPU đa lõi nhưng chi phí tạo luồng cao. Mô hình đang được sử dụng là gì?

a) Many-to-One b) One-to-One
c) Many-to-Many d) Thread Pool

Đáp án: B - One-to-One ánh xạ mỗi luồng người dùng với một luồng hạt nhân, hỗ trợ chạy song song trên đa lõi nhưng chi phí tạo và quản lý luồng cao hơn.

32. Trong một hệ thống, nếu một luồng người dùng bị chặn I/O thì toàn bộ các luồng khác trong cùng tiến trình cũng bị chặn theo. Hệ thống này đang sử dụng mô hình nào?

a) Many-to-One
b) One-to-One
c) Many-to-Many
d) Hybrid Scheduler

Đáp án: A - Many-to-One ánh xạ nhiều luồng người dùng vào một luồng hạt nhân, nên khi một luồng bị chặn thì toàn bộ tiến trình bị chặn theo.

33. Một hệ điều hành cho phép ánh xạ nhiều luồng người dùng lên nhiều luồng hạt nhân một cách linh hoạt, giúp vừa tăng hiệu năng vừa tránh giới hạn số lượng luồng hạt nhân. Mô hình này là gì?

a) Many-to-One
b) One-to-One
c) Many-to-Many
d) Single-threaded model

Đáp án: C - Many-to-Many cho phép ánh xạ linh hoạt nhiều luồng người dùng lên nhiều luồng hạt nhân, tối ưu hiệu năng và khả năng mở rộng.

34. Trong hệ điều hành Windows, mô hình luồng được sử dụng phổ biến là gì?

a) Many-to-One, trong đó nhiều luồng người dùng dùng chung một luồng hạt nhân.
b) One-to-One, trong đó mỗi luồng người dùng ánh xạ với một luồng hạt nhân.
c) Many-to-Many, cho phép ánh xạ linh hoạt giữa user thread và kernel thread.
d) Không sử dụng luồng hạt nhân, chỉ dùng luồng người dùng.

Đáp án: B - Windows sử dụng mô hình One-to-One, trong đó mỗi luồng người dùng được ánh xạ trực tiếp với một luồng hạt nhân, giúp tăng khả năng song song trên hệ thống đa lõi.

35. Chọn phát biểu đúng về *Thread Pool* và *Fork-Join*:

a) Thread Pool tạo thread mới mỗi khi có công việc đến để tăng tính linh hoạt.
b) Fork-Join chỉ phù hợp với các chương trình đơn luồng vì cần đồng bộ kết quả.
c) Thread Pool giúp tái sử dụng thread có sẵn, còn Fork-Join chia nhỏ công việc rồi hợp nhất kết quả.
d) Cả Thread Pool và Fork-Join đều yêu cầu mỗi tác vụ phải chạy tuần tự.

Đáp án: C

36. Chọn phương pháp KHÔNG thuộc implicit threading:

a) Intel Threading Building Blocks
b) Grand Central Dispatch
c) OpenMP
d) Process Scheduling

Đáp án: D - Process Scheduling

37. Phát biểu nào sau đây đúng về hai cơ chế lập lịch CPU *Preemptive* và *Non-preemptive*?

a) Preemptive là khi tiến trình tự nguyện nhả CPU, còn Non-preemptive là khi hệ thống tước CPU của tiến trình.
b) Non-preemptive là khi tiến trình giữ CPU cho đến khi kết thúc hoặc tự chờ I/O, còn Preemptive cho phép hệ điều hành tước CPU của tiến trình.
c) Cả hai đều không cho phép hệ điều hành can thiệp vào CPU.
d) Preemptive chỉ dùng cho hệ thống đơn nhiệm.

Đáp án: B - Non-preemptive scheduling: tiến trình tự nhả CPU khi kết thúc hoặc chờ I/O. Preemptive scheduling: hệ điều hành có thể tước CPU của tiến trình để cấp cho tiến trình khác.

38. Phát biểu nào sau đây đúng về *Dispatch Latency* trong hệ điều hành?

a) Là thời gian CPU thực thi một tiến trình từ đầu đến cuối.
b) Là thời gian hệ điều hành cần để tạo một tiến trình mới.
c) Là thời gian để bộ lập lịch dừng một tiến trình và chuyển CPU sang tiến trình khác.
d) Là thời gian truy xuất dữ liệu từ bộ nhớ chính.

Đáp án: C - Dispatch latency là thời gian mà dispatcher cần để dừng tiến trình đang chạy, thực hiện context switch và bắt đầu tiến trình khác chạy trên CPU.

39. Cho 3 tiến trình sau (đơn vị: giây):

Process	Arrival Time	Burst Time
P1	0	5
P2	1	6
P3	2	3

Áp dụng thuật toán lập lịch *FCFS (First Come First Serve)*, hãy tính **Average Waiting Time** của các tiến trình.

a) 3.00 b) 4.33 c) 4.00 d) 5.00

Đáp án: B - First-Come, First-Served (FCFS): Tiến trình nào đến trước sẽ được phục vụ trước theo cơ chế FIFO.

$$Gantt: P1(0-5) \rightarrow P2(5-11) \rightarrow P3(11-14)$$

Average Waiting Time: $(0+4+9)/3 = 10/3 = 4.33$

40. Cho 3 tiến trình sau (đơn vị: giây):

Process	Arrival Time	Burst Time
P1	0	5
P2	1	6
P3	2	3

Áp dụng thuật toán lập lịch *SJF (Shortest Job First - Non-preemptive)*, hãy tính **Average Waiting Time** của các tiến trình.

a) 2.33 b) 3.00 c) 3.67 d) 4.33

Đáp án: C - SJF (Non-preemptive): luôn chọn tiến trình có Burst Time nhỏ nhất trong hàng đợi Ready.

$$Gantt: P1(0-5) \rightarrow P3(5-8) \rightarrow P2(8-14)$$

Average Waiting Time: $\frac{0+3+7}{3} = \frac{10}{3} \approx 3.33$.

41. Cho 3 tiến trình sau (đơn vị: giây) và thuật toán lập lịch *Round Robin (RR)* với quantum = 2:

Process	Arrival Time	Burst Time
P1	0	5
P2	1	6
P3	2	3

Hãy tính **Average Waiting Time** của các tiến trình.

a) 3.33 b) 4.00 c) 4.67 d) 5.33

Đáp án: B - Round Robin (quantum = 2): các tiến trình được chia CPU theo lát thời gian công bằng.

$$Gantt: P1(0-2) \rightarrow P2(2-4) \rightarrow P3(4-6) \rightarrow P1(6-8) \rightarrow P2(8-10) \rightarrow P3(10-11) \rightarrow P1(11-12) \rightarrow P2(12-14)$$

Average Waiting Time: $\frac{7+7+6}{3} = \frac{20}{3} \approx 6.67$

42. Cho 4 tiến trình sau (đơn vị: ms):

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Áp dụng thuật toán *SRTF (Shortest Remaining Time First - Preemptive SJF)*, hãy tính **Average Turnaround Time**.

a) 10.25 b) 11.50 c) 13.00 d) 13.25

Đáp án: C - https://geekcandy.co.uk/scheduling.html

43. Xét các phát biểu về việc áp dụng các giải thuật định thời CPU trong thực tế:

(1) FCFS phù hợp cho các hệ thống xử lý bó (batch processing).

(2) Round Robin thường được dùng cho các hệ thống tương tác vì cải thiện response time.

(3) Priority Scheduling thường xuất hiện trong hệ điều hành thời gian thực.

(4) Multilevel Feedback Queue được sử dụng trong nhiều hệ điều hành hiện đại do có thể thích nghi với nhiều loại workload.

Có bao nhiêu phát biểu đúng?

a) 1 b) 2 c) 3 d) 4

Đáp án: D

44. Chọn phát biểu đúng về ảnh hưởng của *Quantum Time* trong giải thuật Round Robin:

a) Quantum quá nhỏ giúp giảm số lần chuyển ngữ cảnh và tăng CPU utilization.
b) Quantum quá lớn làm Round Robin hoạt động gần giống FCFS.
c) Quantum càng nhỏ thì response time càng tăng.
d) Quantum không ảnh hưởng đến hiệu năng của Round Robin.

Đáp án: B

45. Xét hai process cùng thao tác trên biến dùng chung `counter`:

Producer:

```
counter++;
```

Consumer:

```
counter--;
```

Biết rằng mỗi phép toán tăng/giảm không thực hiện nguyên tử (atomic) mà gồm nhiều bước đọc, xử lý và ghi dữ liệu. Phát biểu nào sau đây đúng?

a) Giá trị cuối cùng của `counter` luôn không đổi.
b) Có thể xảy ra dữ liệu không nhất quán do race condition.
c) Hệ điều hành luôn tự động đồng bộ hai process.
d) Race condition chỉ xảy ra trên hệ thống đa lõi.

Đáp án: B - Vì các thao tác tăng/giảm không nguyên tử nên việc xen kẽ thực thi có thể làm mất cập nhật dữ liệu và gây race condition.

46. Xét hai process P_0 và P_1 cùng gọi `fork()` để tạo process con. Hệ điều hành sử dụng biến dùng chung `next_available_pid` để cấp phát PID mới cho các process.

Nếu không có cơ chế *mutual exclusion*, điều gì có thể xảy ra?

a) Hai process con có thể được cấp cùng một PID.
b) Hệ điều hành luôn tự động tránh trùng PID.
c) Một trong hai process sẽ bị deadlock.
d) PID chỉ có thể bị trùng trên hệ thống đa CPU.

Đáp án: A - Nếu nhiều process cùng truy cập và cập nhật biến `next_available_pid` đồng thời mà không đồng bộ, race condition có thể xảy ra và dẫn đến việc cấp trùng PID.

47. Cấu trúc thông thường của một tiến trình trong bài toán *Critical Section* gồm những phần nào?

a) Entry section, Critical section, Exit section, Remainder section.
b) Ready section, Waiting section, Running section, Exit section.
c) User section, Kernel section, Waiting section, Ready section.
d) Input section, Processing section, Output section.

Đáp án: A - Một tiến trình trong bài toán critical-section thường gồm: entry section (xin quyền vào), critical section (truy cập dữ liệu chia sẻ), exit section (rời vùng tranh chấp), và remainder section (phần còn lại của chương trình).

48. Trong các trường hợp sau, trường hợp nào **không** thuộc bài toán *Critical Section*?

a) Nhiều process cùng cập nhật biến đếm dùng chung.
b) Nhiều thread cùng ghi dữ liệu vào một file.
c) Hai process cùng truy cập bảng cấp phát bộ nhớ.
d) Một process thực hiện tính toán trên biến cục bộ riêng của nó.

Đáp án: D - Critical-section chỉ xảy ra khi có dữ liệu hoặc tài nguyên dùng chung giữa nhiều process/thread. Biến cục bộ riêng không gây race condition.

49. Để giải quyết bài toán *Critical Section*, một lời giải đúng cần thoả mãn những tính chất nào sau đây?

a) Mutual Exclusion, Progress, Bounded Waiting.
b) Deadlock Prevention, Paging, Scheduling.
c) Starvation, Context Switching, Mutual Exclusion.
d) Progress, Dispatching, Swapping.

Đáp án: A - Ba tính chất quan trọng:

- ***Mutual Exclusion**: Không có hai process cùng ở critical section.*
- ***Progress**: Nếu critical section rỗng thì process muốn vào phải được chọn trong thời gian hữu hạn.*
- ***Bounded Waiting**: Có giới hạn số lần process khác được vào trước khi yêu cầu hiện tại được chấp nhận.*

50. Phát biểu nào đúng về xử lý *Critical Section* trong hệ điều hành?

a) Kernel preemptive không cho phép process bị chiếm quyền khi đang ở kernel mode.
b) Kernel non-preemptive chạy đến khi thoát kernel mode, block hoặc tự nhường CPU.
c) Kernel preemptive luôn không có race condition.
d) Kernel non-preemptive cho phép process khác chiếm CPU bất kỳ lúc nào trong kernel mode.

Đáp án: B - Trong non-preemptive kernel, process đang chạy trong kernel mode sẽ không bị preempt cho đến khi thoát kernel mode, block hoặc voluntarily yield CPU.

51. Xét lời giải đồng bộ sử dụng biến dùng chung `turn`:

$$\texttt{turn} = i \Rightarrow P_i \text{ được phép vào critical section}$$

Phát biểu nào sau đây đúng về lời giải này?

a) Thoả cả Mutual Exclusion và Progress.
b) Vi phạm Mutual Exclusion nhưng thoả Progress.
c) Thoả Mutual Exclusion nhưng vi phạm Progress.
d) Vi phạm cả Mutual Exclusion và Bounded Waiting.

*Đáp án: C - dùng biến `turn` đảm bảo tại một thời điểm chỉ một process vào critical section nên thoả **Mutual Exclusion**. Tuy nhiên, nếu một process không muốn vào critical section nhưng vẫn giữ lượt (`turn`) của mình thì process còn lại phải chờ vô hạn, nên vi phạm **Progress**.*

52. Xét hai process sau sử dụng biến dùng chung `turn`:

Process P_0

```
do {
    while (turn != 0);
    critical section
    turn = 1;
    remainder section
} while (1);
```

Process P_1

```
do {
    while (turn != 1);
    critical section
    turn = 0;
    remainder section
} while (1);
```

Trường hợp nào sau đây làm một process phải *busy waiting* vô hạn mặc dù critical section đang rỗng?

a) *turn* = 0, P_0 ở remainder section rất lâu và P_1 muốn vào critical section.
b) *turn* = 1, cả hai process cùng muốn vào critical section.
c) *turn* được thay đổi sau mỗi lần vào critical section.
d) Cả hai process đều liên tục vào critical section.

Đáp án: A - Nếu `turn = 0` nhưng P_0 không muốn vào critical section, P_1 vẫn phải chờ ở vòng lặp `while(turn != 1);` dù không ai ở critical section, dẫn đến vi phạm progress.

53. Xét lời giải đồng bộ sử dụng mảng cờ:

$$flag[i] = true$$

để báo rằng process P_i muốn vào *critical section*.

Phát biểu nào sau đây đúng?

a) Có thể xảy ra trường hợp cả P_0 và P_1 cùng chờ vô hạn.
b) Hai process có thể đồng thời vào critical section.
c) Lời giải thoả cả Mutual Exclusion và Progress.
d) Lời giải không sử dụng busy waiting.

*Đáp án: A - Nếu cả hai process cùng thực hiện: $flag[0] = true$, $flag[1] = true$ Cả hai sẽ chờ nhau vô hạn dù không process nào ở critical section, nên vi phạm điều kiện **Progress**.*

54. Xét đoạn chương trình sau:

```
while (true){
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);

    /* critical section */

    flag[i] = false;

    /* remainder section */
}
```

Đoạn chương trình trên là lời giải nào cho bài toán *Critical Section* và có thoả mãn các tính chất nào?

a) Giải thuật Peterson, chỉ thoả Mutual Exclusion.
b) Giải thuật Peterson, thoả cả Mutual Exclusion, Progress và Bounded Waiting.
c) Giải thuật Dekker, chỉ thoả Progress.

d) Giải thuật Test-and-Set, không thoả Bounded Waiting.

Đáp án: B - Dây là thuật toán dành cho 2 tiến trình chia sẻ hai biến là turn và flag[]. Thuật toán này đáp ứng cả 3 điều kiện của miền găng, nhưng không được đảm bảo sẽ hoạt động đúng trên các kiến trúc máy tính hiện đại do trình biên dịch hoặc bộ xử lý có thể sắp xếp lại các lệnh

55. Xét hai process P_0 và P_1 sử dụng Peterson's Solution:

```
while (true){
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);

    /* critical section */

    flag[i] = false;

    /* remainder section */
}
```

Giả sử ban đầu:

$$flag[0] = flag[1] = false$$

và các bước thực thi xảy ra theo thứ tự:

$$S1 : P_0 \rightarrow flag[0] = true$$
$$S2 : P_1 \rightarrow flag[1] = true$$
$$S3 : P_1 \rightarrow turn = 0$$
$$S4 : P_0 \rightarrow turn = 1$$
$$S5 : P_0 \rightarrow while$$
$$S6 : P_1 \rightarrow while$$

Sau bước $S6$, process nào sẽ được vào *critical section* trước?

a) Chỉ P_0 b) Chỉ P_1 c) Cả hai cùng vào d) Cả hai bị chặn

Đáp án: B - ở S5 đang Trạng thái các biến dùng chung hiện tại: flag[0] = true, flag[1] = true, turn = 1.

56. Vì sao Peterson's Solution có thể không hoạt động chính xác trên các kiến trúc hiện đại?

a) Vì hệ điều hành không hỗ trợ mutual exclusion.
b) Vì compiler hoặc CPU có thể thực hiện instruction reordering.
c) Vì Peterson's Solution chỉ dùng được cho một process.
d) Vì biến `flag` không thể dùng trong đa luồng.

Đáp án: B - Để tối ưu hiệu năng, CPU hoặc compiler có thể đổi thứ tự các lệnh độc lập (instruction reordering). Trong chương trình đa luồng, việc đổi thứ tự này có thể làm các thread quan sát dữ liệu không nhất quán, khiến Peterson's Solution không còn đảm bảo mutual exclusion trên kiến trúc hiện đại.

57. Vai trò của `memory_barrier()` trong chương trình đa luồng là gì?

a) Tăng tốc độ thực thi của thread bằng cách bỏ qua đồng bộ.
b) Ngăn CPU hoặc compiler đổi thứ tự các thao tác bộ nhớ quan trọng.
c) Cho phép nhiều thread cùng vào critical section.
d) Thay thế hoàn toàn semaphore và mutex.

Đáp án: B - `memory_barrier()` được dùng để ngăn hiện tượng instruction reordering của CPU/compiler. Nhờ đó các thao tác đọc/ghi bộ nhớ sẽ được nhìn thấy theo đúng thứ tự mong muốn giữa các thread, giúp tránh dữ liệu không nhất quán trong chương trình đa luồng.

58. Chọn phát biểu đúng về hai lệnh đồng bộ phần cứng `test_and_set()` và `compare_and_swap()`:

    a) Cả hai lệnh đều được thực thi nguyên tử (atomic).
    b) `test_and_set()` chỉ đọc dữ liệu mà không thay đổi giá trị.
    c) `compare_and_swap()` luôn thay đổi giá trị của biến.
    d) Hai lệnh trên không thể dùng để hiện thực mutual exclusion.

    Đáp án: A - Cả hai thường được dùng để xây dựng spinlock và cơ chế mutual exclusion.

    - *`test_and_set()`: đọc giá trị cũ của biến lock rồi đặt lock thành true một cách atomic.*
    - *`compare_and_swap()`: chỉ thay đổi giá trị nếu giá trị hiện tại đúng bằng giá trị kỳ vọng.*

59. Phát biểu nào mô tả đúng nhất về đặc tính của lệnh `Compare-And-Swap (CAS)` trong cơ chế đồng bộ?

    a) CAS đọc dữ liệu và cập nhật dữ liệu thành nhiều bước độc lập để tăng hiệu suất.
    b) CAS đảm bảo thao tác kiểm tra và cập nhật dữ liệu được thực hiện như một thao tác không thể bị xen giữa.
    c) CAS chỉ hoạt động đúng trên hệ thống đơn lõi (single-core).
    d) CAS yêu cầu semaphore để đảm bảo tính đúng đắn.

    Đáp án: B - Tính nguyên tử (Atomicity): Đặc tính cốt lõi của lệnh Compare-And-Swap (CAS) là nó thực hiện việc kiểm tra (compare) và cập nhật (swap) dữ liệu dưới dạng một thao tác nguyên tử ở mức phần cứng. Nghĩa là toàn bộ quá trình này diễn ra trọn vẹn trong một bước, không có bất kỳ luồng (thread) hay tiến trình (process) nào khác có thể chen ngang hoặc ngắt quãng, qua đó ngăn chặn tình trạng tương tranh (race condition).

60. Phát biểu nào đúng về *atomic variable*?

    a) Atomic variable cho phép nhiều thread cập nhật dữ liệu mà không cần đồng bộ.
    b) Atomic variable đảm bảo thao tác cập nhật dữ liệu diễn ra không bị ngắt quãng.
    c) Atomic variable chỉ dùng được với kiểu dữ liệu chuỗi ký tự.
    d) Atomic variable luôn cần semaphore để hoạt động đúng.

    Đáp án: B

61. Chọn phát biểu đúng về *mutex lock* trong cơ chế đồng bộ:

    a) Mutex lock loại bỏ hoàn toàn busy waiting.
    b) Các thao tác `acquire()` và `release()` không cần thực hiện atomic.
    c) Mutex lock sử dụng busy waiting nên còn được gọi là spinlock.
    d) Mutex lock cho phép nhiều process cùng vào critical section nếu CPU đủ mạnh.

    Đáp án: C - Mutex lock hoạt động bằng cách: acquire() → critical section → release() Nếu lock chưa khả dụng, process sẽ liên tục kiểm tra: while(!available) đây là busy waiting, và vậy mutex dạng này còn được gọi là ***spinlock****.*

62. Đoạn mã sau đang hiện thực cơ chế đồng bộ nào?

```
boolean available = true;

acquire() {
    while (!available)
        ; /* busy waiting */
    available = false;
}

release() {
    available = true;
}
```

```
while (true) {
    acquire();

    critical section;

    release();

    remainder section;
}
```

a) Semaphore counting
b) Peterson's Solution
c) Mutex Lock
d) Monitor

Đáp án: C

63. Cho hai process sử dụng semaphore như sau:

```
Process P1:
while (1) {
    wait(S1);

    Critical Section;

    signal(S2);
}

Process P2:
while (1) {
    wait(S2);

    Critical Section;

    signal(S1);
}

S1 = 1;
S2 = 0;
```

Thứ tự thực thi các *Critical Section* sẽ là:

a) P1, P1, P2, P2, ...
b) P1, P2, P1, P2, ...
c) P2, P1, P2, P1, ...
d) Không xác định

Đáp án: B - Tiến trình P_2 thực thi wait(S2). Vì $S_2 = 0$, P_2 không thể đi tiếp mà bị chặn lại (block) và đưa vào hàng đợi chờ. Cùng lúc đó, tiến trình P_1 thực thi wait(S1). Vì $S_1 = 1$, thao tác này thành công. S_1 bị giảm xuống 0.

64. Chọn phát biểu đúng về *Counting Semaphore* và *Binary Semaphore*:

a) Counting semaphore chỉ nhận giá trị 0 hoặc 1, còn binary semaphore nhận mọi số nguyên dương.
b) Binary semaphore chỉ nhận giá trị 0 hoặc 1, còn counting semaphore có thể nhận nhiều giá trị nguyên khác nhau.
c) Counting semaphore và binary semaphore hoàn toàn giống nhau.
d) Binary semaphore được dùng để đếm số process đang chạy, còn counting semaphore dùng cho mutual exclusion.

Đáp án: B

65. Trường hợp nào sau đây có thể xảy ra khi sử dụng semaphore không đúng cách?

a) Gọi `signal(mutex)` trước `wait(mutex)` có thể làm mất tính đồng bộ.

b) Gọi `wait(mutex)` hai lần liên tiếp có thể gây deadlock.
c) Bỏ quên `wait()` hoặc `signal()` có thể làm chương trình hoạt động sai.
d) Tất cả các ý trên đều đúng.

Đáp án: D

66. Chọn phát biểu đúng về *Monitor* và *Condition Variable*:

a) Nhiều process có thể đồng thời thực thi bên trong cùng một monitor.
b) Lời gọi `x.signal()` luôn làm process gọi bị block ngay lập tức.
c) Nếu không có process nào đang chờ tại `x.wait()`, lời gọi `x.signal()` sẽ không có tác dụng.
d) Condition variable thay thế hoàn toàn mutex và mutual exclusion.

Đáp án: C

67. [HK242] Giải thuật nào sau đây có thời gian đợi trung bình tốt nhất trong hầu hết các trường hợp?

a) FCFS b) SRTF c) SJF d) Round-robin

Đáp án: B - Sự linh hoạt của SRTF (Preemptive SJF): Trong môi trường thực tế, các tiến trình liên tục sinh ra và đi vào hàng đợi ở các thời điểm khác nhau. SRTF giải quyết được bài toán này. Nếu một tiến trình mới đến có thời gian cần CPU (burst time) ngắn hơn thời gian còn lại của tiến trình đang chạy, SRTF sẽ lập tức ngắt (preempt) tiến trình hiện tại để nhường CPU cho tiến trình mới.

68. [HK242] Thuật toán lập lịch nào phù hợp nhất cho các hệ điều hành thời gian thực?

a) FCFS b) Định thời ưu tiên c) SJF d) Round Robin

Đáp án: B - Trong hệ điều hành thời gian thực, các tiến trình thường có mức độ khẩn cấp khác nhau và phải hoàn thành trước deadline. Định thời ưu tiên cho phép CPU phục vụ ngay các tiến trình có độ ưu tiên cao, giúp đảm bảo yêu cầu thời gian thực tốt hơn so với FCFS, SJF hay Round Robin.

69. [HK242] Giải thuật định thời CPU là Priority và preemptive với giá trị Priority nhỏ hơn có độ ưu tiên cao hơn, hãy tính thời gian chờ trung bình (avg. waiting time) của các quá trình thực thi. Cho bảng thông tin các quá trình gồm (Burst time tính bằng giây):

Process	Arrival time	Burst time	Priority
P0	0	8	4
P1	2	2	2
P2	4	4	2
P3	5	1	1
P4	6	3	1

a) 1, 5. b) 6, 4. c) 2, 8. d) Số khác.

Đáp án: C -

- *P0: Bị ngắt ở thời điểm 2, mãi đến 12 mới được chạy lại → Chờ: 12 − 2 = 10.*
- *P1: Đến lúc 2, chạy ngay lập tức và không bị ngắt → Chờ: 0.*
- *P2: Đến lúc 4, chạy được 1 giây thì bị ngắt lúc 5. Đến 9 được chạy lại → Chờ: 9 − 5 = 4.*
- *P3: Đến lúc 5, chạy ngay và không bị ngắt → Chờ: 0.*
- *P4: Đến lúc 6, chạy ngay và không bị ngắt → Chờ: 0.*

70. [HK242] Trong thuật toán Round Robin, yếu tố nào ảnh hưởng trực tiếp đến hiệu suất định thời?

a) Số lượng quá trình thực thi đang trong hàng chờ.
b) Tốc độ của CPU.
c) Số hàng chờ phân theo độ ưu tiên.
d) Thời gian một quá trình được thực thi trên CPU.

Đáp án: D - Nếu Time Quantum quá lớn: Thuật toán Round Robin sẽ thoái hóa và hoạt động gần giống hệt như thuật toán FCFS (First Come First Serve). Các tiến trình sẽ giữ CPU cho đến khi chạy xong, làm mất đi tính tương tác và thời gian đáp ứng (response time) sẽ kém đi đối với các tiến trình ngắn. Nếu Time Quantum quá nhỏ: Hệ điều hành sẽ phải liên tục thực hiện việc ngắt và chuyển đổi qua lại giữa các tiến trình. Quá trình này gọi là Chuyển ngữ cảnh (Context Switching). Việc chuyển ngữ cảnh tốn một khoảng thời gian nhất định để lưu và nạp trạng thái thanh ghi, bộ nhớ. Nếu Time Quantum quá nhỏ, thời gian CPU dùng để chuyển ngữ cảnh sẽ chiếm tỷ lệ lớn so với thời gian thực sự dùng để xử lý công việc, dẫn đến hiệu suất tổng thể của hệ thống giảm sút nghiêm trọng.

71. [HK242] Giải thuật định thời CPU là SRTF, hãy tính thời gian xoay vòng trung bình (avg. turnaround time) của các quá trình thực thi. Cho bảng thông tin các quá trình gồm (Burst time tính bằng giây):

Process	Arrival time	Burst time (giây)
P0	1	10
P1	2	4
P2	3	1
P3	4	6
P4	6	2

a) 4, 6. b) 6, 4. c) Số khác. d) 8, 6.

Đáp án: D - `https://geekcandy.co.uk/scheduling.html`

72. [HK242] Giải thuật định thời nào sau đây không có tình trạng trì hoãn vô hạn định?

a) SJF. b) SRTF. c) Priority scheduling. d) FCFS.

Đáp án: D - Tình trạng trì hoãn vô hạn định (indefinite postponement) hay còn gọi là chết đói (starvation) xảy ra khi một tiến trình nằm trong hàng đợi chờ CPU nhưng không bao giờ được cấp phát CPU để chạy, do hệ điều hành liên tục ưu tiên các tiến trình khác mới đến.

73. [HK242] Chiến lược sử dụng biến đơn nguyên (atomic variable) phù hợp cho ngữ cảnh nào dưới đây?

a) Nhiều luồng thực thi cùng tải về các phần khác nhau của một tập tin từ Internet.
b) Một chương trình phân tán đếm số lần xuất hiện của các từ trong một văn bản.
c) Các công việc song song yêu cầu thứ tự thực thi phải được đảm bảo.
d) Một web server thống kê số lượng yêu cầu truy cập tới hệ thống.

Đáp án: D - Một web server thường sử dụng mô hình đa luồng (multi-threading) để xử lý đồng thời nhiều yêu cầu (requests) từ người dùng. Việc đếm tổng số yêu cầu truy cập đòi hỏi tất cả các luồng phải cùng tăng giá trị của một biến counter dùng chung. Thao tác counter++ thông thường bao gồm 3 bước (đọc - tăng - ghi) nên rất dễ bị sai lệch số liệu nếu không đồng bộ. Sử dụng biến đơn nguyên (atomic integer) cho bộ đếm này là giải pháp hoàn hảo, vừa đảm bảo tính chính xác tuyệt đối vừa giữ được hiệu suất xử lý cao của web server.

74. [HK242] Trong mô hình lập trình luồng thực thi không tường minh, chọn phát biểu đúng về ưu điểm của Thread Pool.

a) Tạo một luồng thực thi mới khi người dùng gửi yêu cầu hàm thực thi và kết thúc khi hoàn tất.
b) Tạo một nhóm các luồng thực thi dựa trên tỷ lệ quy mô phần cứng và kích thước bài toán và ánh xạ theo tỷ lệ để thực thi công việc.
c) Tạo một số lượng luồng thực thi và chỉ gán tác vụ khi người dùng gửi yêu cầu để giảm thời gian mỗi lần tạo luồng.
d) Sử dụng hàng đợi FIFO để quản lý và lần lượt ánh xạ các tác vụ theo hàng đợi vào luồng tương ứng để thực thi.

Đáp án: C - Thread Pool (Hồ chứa luồng): Là một kỹ thuật quản lý luồng trong hệ điều hành (thuộc nhóm implicit threading - lập trình luồng không tường minh do compiler hoặc thư viện quản lý). Ý tưởng cốt lõi là tạo sẵn một số lượng luồng nhất định (pool) ngay từ khi tiến trình khởi động và để chúng ở trạng thái chờ (waiting).

75. [HK242] Cấu trúc monitor đảm bảo rằng

a) N quá trình có thể thực thi trong monitor tại cùng một thời điểm (N > 1).
b) Hàng đợi chỉ có một quá trình thực thi tại một thời điểm cụ thể.
c) Tại một thời điểm chỉ có thể có một quá trình thực thi trong monitor.
d) Các quá trình chờ tại một rào chắn (barrier) để thực hiện cùng lúc.

Đáp án: C - Monitor (Giám sát): Là một cấu trúc đồng bộ hóa cấp cao trong hệ điều hành và ngôn ngữ lập trình. Nó đóng gói các biến dữ liệu chia sẻ (shared data) và các thủ tục (procedures/methods)

thao tác trên dữ liệu đó vào trong một mô-đun duy nhất.

76. [HK242] Câu nào sau đây không đúng về giao tiếp truyền thông điệp?

a) Trong blocking receive, bên nhận bị khóa cho đến khi nhận được một thông điệp.
b) Trong non-blocking send, bên gửi truyền thông điệp và tiếp tục.
c) Trong non-blocking receive, bên nhận nhận một thông điệp hợp lệ hoặc rỗng.
d) Trong blocking send, bên gửi bị khóa cho đến khi thông điệp được gửi.

Đáp án: D - Blocking send (Gửi đồng bộ) là tiến trình gửi sẽ bị khóa (blocked) cho đến khi thông điệp được nhận (received) bởi tiến trình đích hoặc được đưa vào hộp thư (mailbox) một cách an toàn.

77. [HK242] Trong cơ chế loại trừ tương hỗ, việc sử dụng cơ chế spinlock có thời gian chờ quá lâu sẽ gây ảnh hưởng như thế nào?

a) Tác vụ lãng phí thời gian được cấp CPU làm giảm hiệu suất sử dụng hệ thống.
b) Tác vụ được chuyển sang trạng thái sleep.
c) Tác vụ bị tạm dừng và swap out ra khỏi hệ thống.
d) Tác vụ phải tiếp tục chờ trong hàng đợi các quá trình bị nghẽn.

Đáp án: A - Spinlock (Khóa quay): Đúng như tên gọi, khi một tiến trình/luồng cố gắng lấy một spinlock đang bị giữ bởi luồng khác, nó sẽ không đi ngủ (sleep) hay chặn (block). Thay vào đó, nó liên tục chạy một vòng lặp rỗng (vòng lặp kiểm tra điều kiện) ngay trên CPU để xem khóa đã được nhả ra chưa. Trạng thái này gọi là busy wasting (chờ bận).

78. [HK242] Cơ chế nào giúp thay đổi trạng thái của một quá trình thực thi?

a) Thực thi lời gọi hệ thống kill để gửi signal.
b) Giao tiếp qua cơ chế giao tiếp giữa các quá trình thực thi (IPC).
c) Xử lý đồng bộ bằng mutex.
d) Khởi tạo tiến trình bằng execv.

Đáp án: A - Lời gọi hệ thống kill(): Mặc dù tên gọi là "kill", system call này thực chất được dùng để gửi bất kỳ tín hiệu nào đến một tiến trình (chứ không chỉ để "giết" tiến trình). Thông qua việc gửi các signal khác nhau, hệ điều hành hoặc người dùng có thể ép buộc tiến trình thay đổi trạng thái hiện tại của nó ngay lập tức

79. [HK242] Vì sao cần có kỹ thuật giao tiếp để chia sẻ dữ liệu giữa các quá trình (IPC) nhưng không cần trong giao tiếp giữa các luồng thread?

a) Vì luồng là một hiện thực gọn nhẹ lightweight của process.
b) Vì luồng hỗ trợ cơ chế tạo một tập hợp pooling theo nhóm.
c) Vì luồng có cơ chế đồng bộ giữa các chồng thực thi stack.
d) Vì luồng dùng chung đường truy xuất bộ nhớ memory bus.

Đáp án: D - Luồng (Thread): Các luồng nằm trong cùng một tiến trình sẽ chia sẻ chung không gian địa chỉ bộ nhớ của tiến trình đó (đặc biệt là vùng Data chứa biến toàn cục và vùng Heap chứa bộ nhớ cấp phát động)

80. [HK242] Thành phần nào KHÔNG được chia sẻ giữa các luồng thực thi (thread) trong một quá trình thực thi đa luồng?

a) Code. b) Data. c) Stack. d) Files.

Đáp án: C - Stack (Ngăn xếp): Mỗi luồng thực hiện một chuỗi các lời gọi hàm (function calls) khác nhau và có các biến cục bộ (local variables) riêng. Do đó, hệ điều hành bắt buộc phải cấp phát cho mỗi luồng một vùng Stack hoàn toàn độc lập.

81. [HK242] Lệnh CompareAndSwap (CAS) được thực thi

a) Có thể bị ngắt (interrupted). b) Định kỳ.
c) Có thể bị chia nhỏ thành nhiều CPU instruction. d) Đơn nguyên (atomic).

Đáp án: D - CompareAndSwap (CAS) là là một lệnh phần cứng (hardware instruction) đặc biệt được cung cấp bởi kiến trúc CPU hiện đại nhằm hỗ trợ đồng bộ hóa các luồng/tiến trình (synchronization). Chức năng của nó là: kiểm tra xem giá trị tại một địa chỉ bộ nhớ có bằng với một "giá trị kỳ vọng" hay không; nếu bằng, nó sẽ cập nhật địa chỉ đó bằng một "giá trị mới".

82. [HK242] Mô hình nào thích hợp hơn cho việc truyền dữ liệu giữa các quá trình thực thi đang chạy trên các CPU khác nhau?

    a) Truyền thông điệp.
    b) Bộ nhớ chia sẻ.
    c) Bộ nhớ BIOS.
    d) Thiết bị lưu trữ USB.

Đáp án: A - Trong giao tiếp liên tiến trình (IPC - Inter-Process Communication), có hai mô hình chính là Truyền thông điệp (Message Passing) và Bộ nhớ chia sẻ (Shared Memory)

83. [HK242] Stack của luồng thực thi và stack của quá trình thực thi được lưu ở đâu?

    a) Trong mã thực thi của chương trình.
    b) Trong phân đoạn dữ liệu data segment.
    c) Trong bộ nhớ chính của hệ thống.
    d) Trong thanh bộ thanh ghi của CPU.

Đáp án: D - Vị trí lưu trữ: Vùng không gian này bắt buộc phải nằm trong bộ nhớ chính (Main Memory hay RAM) của máy tính.

84. [HK242] Một lệnh máy được coi là đơn nguyên khi nào?

    a) Khi lệnh đó được thực thi trong 1 clock cycle duy nhất.
    b) Khi lệnh đó được sử dụng cho tất cả các lời gọi hệ thống trong hệ điều hành.
    c) Khi lệnh đó dùng cho cơ chế đồng bộ hoá.
    d) Khi lệnh đó không bị ngắt khi đang thực thi.

Đáp án: D

85. [HK242] Tất cả các quá trình thực thi đều dùng chung 1 semaphore mutex với giá trị khởi tạo là 1. Các quá trình đều thực thi đoạn mã nguồn dưới đây. Hãy chọn câu phát biểu đúng?

```
semaphore mutex S
...
signal( S )
//critical section
wait( S )
...
```

    a) Nhiều quá trình có thể thực thi vùng tranh chấp tại cùng một thời điểm.
    b) Tắc nghẽn (deadlock) sẽ xảy ra.
    c) Có thể xảy ra tình trạng trì hoãn vô hạn định.
    d) Việc thực thi được đảm bảo loại trừ tương hỗ.

Đáp án: A -

- *Ban đầu Semaphore S = 1.*
- *Giả sử Tiến trình 1 (P1) chạy đến, nó gọi signal(S). Lệnh signal lập tức tăng giá trị S lên thành 2. Sau đó P1 đi vào vùng tranh chấp.*
- *Ngay lúc này, Tiến trình 2 (P2) cũng chạy đến. Nó tiếp tục gọi signal(S). Giá trị S tăng lên thành 3. Vì không có lệnh wait nào chặn lại ở đầu, P2 cũng thản nhiên bước vào vùng tranh chấp.*
- *Cứ như vậy, bất kỳ tiến trình nào tới cũng sẽ tăng S lên và đi thẳng vào trong.*

86. [HK242] Yêu cầu tiến trình cho một giải thuật đồng bộ hoá trên vùng tài nguyên bị tranh chấp có nghĩa là?

a) Chỉ có một quá trình thực thi duy nhất được truy cập vào vùng tài nguyên bị tranh chấp.
b) Nếu một quá trình thực thi đang chờ đã có số lần yêu cầu truy cập vùng tài nguyên bị tranh chấp quá một ngưỡng nào đó thì sẽ được ưu tiên khi vùng đó rảnh rỗi.
c) Khi vùng tài nguyên bị tranh chấp vừa rảnh rỗi, một trong các quá trình thực thi đang chờ có thể truy cập ngay
d) Một quá trình thực thi truy cập vùng tài nguyên bị tranh chấp phải được đảm bảo không bị gián đoạn cho tới khi quá trình thực thi đó hoàn thành.

Đáp án: C

87. [HK242] Cơ chế nào gây ra sai sót khi hiện thực giải thuật Peterson's Lock?

a) Bounded waiting.
b) Mutual exclusion.
c) Instruction reordering
d) Progress.

Đáp án: C - Giải thuật Peterson (Peterson's Solution): Là một giải pháp phần mềm kinh điển để giải quyết bài toán miền găng (critical section) cho 2 tiến trình. Về mặt lý thuyết toán học, giải thuật này hoàn toàn đúng và thỏa mãn cả 3 điều kiện: Mutual exclusion (B), Progress (D), và Bounded waiting (A).

88. [HK242] Cho 2 quá trình thực thi P1 và P2 như mô tả dưới đây. P1 và P2 cùng chạy với giá trị S1 và S2 được khởi tạo ngẫu nhiên. Hãy chọn phát biểu đúng.

P1:	**P2:**
`while(S1==S2);`	`while(S1!=S2);`
`//Critical section`	`//Critical section`
`S1 = S2;`	`S2 = not(S1);`

a) Đảm bảo điều kiện loại trừ tương hỗ (mutual exclusion) nhưng không thoả điều kiện tiến trình (progress).
b) Đảm bảo thoả cả hai điều kiện loại trừ tương hỗ và tiến trình.
c) Đảm bảo điều kiện tiến trình nhưng không thoả điều kiện loại trừ tương hỗ.
d) Các lựa chọn khác đều sai.

Đáp án: B - Loại trừ tương hỗ (Mutual Exclusion): Quá trình P1 chỉ vào miền găng khi S1 ≠ S2, còn P2 chỉ vào khi S1 = S2. Hai điều kiện này hoàn toàn đối lập nhau, nên hai tiến trình không bao giờ có thể cùng lúc ở trong miền găng. Tiến trình (Progress): Tại bất kỳ thời điểm nào, luôn có chính xác một tiến trình thỏa mãn điều kiện để đi vào miền găng. Khi chạy xong, tiến trình đó cập nhật giá trị biến để nhường lượt cho tiến trình kia. Việc này giúp hệ thống luân phiên liên tục, không bao giờ bị kẹt (deadlock) nên đảm bảo được điều kiện tiến trình.

89. [HK242] Lợi ích của cơ chế đa luồng thực thi trên các nền tảng đơn nhân và đa nhân là:

a) Luồng thực thi hỗ trợ cả hai môi trường để chia nhỏ thành các tác vụ độc lập.
b) Luồng thực thi bị giới hạn hỗ trợ trên máy đơn nhân vì không hỗ trợ thực thi đồng thời.
c) Luồng thực thi hỗ trợ trên nền tảng đa nhân và bị giới hạn chia sẻ giữa các nhân.
d) Luồng thực thi hỗ trợ máy đơn nhân nhưng phải hoàn tất từng tác vụ rồi đến tác vụ kế tiếp.

Đáp án: A

90. [HK242] Trong các cách sau, cách nào không sử dụng các luồng thực thi KHÔNG tường minh (implicit threading)?

a) OpenMP.
b) Intel Threading Building Blocks.
c) Pthread.
d) Grand Central Dispatch.

Đáp án: C - Pthread (POSIX Threads) chính là đại diện kinh điển nhất của luồng tường minh. Khi dùng Pthread, bạn bắt buộc phải gọi trực tiếp các API như pthread_create() để sinh ra một luồng mới, hay pthread_join() để đợi luồng kết thúc.

91. [HK242] Cấu trúc nào sau đây không phải là cấu trúc có riêng của mỗi luồng thực thi?

a) Program counter. b) CPU register. c) Code segment. d) Stack register.

Đáp án: C - Phân đoạn mã (chứa các câu lệnh đã biên dịch của chương trình). Tất cả các luồng thuộc cùng một tiến trình đều dùng chung một bộ mã nguồn, do đó chúng chia sẻ chung Code segment trên bộ nhớ. Đoạn mã này là của chung tiến trình chứ không thuộc sở hữu riêng của luồng nào cả.

92. [HK211] Giải thuật định thời CPU nào dưới đây phù hợp cho các ứng dụng *I/O-bound*?

a) Round Robin với quantum time ngắn b) Shortest Job First
c) FCFS d) SRTF

Đáp án: A - RR với quantum nhỏ giúp các process I/O-bound được cấp CPU thường xuyên hơn, tăng tính đáp ứng và hiệu quả tương tác.

93. [HK211] Cơ chế ánh xạ user thread vào kernel thread nào sau đây giúp tránh tình trạng "một thread bị block thì tất cả thread khác cũng bị ảnh hưởng"?

a) One-to-one và many-to-many b) Many-to-one và many-to-many
c) Các cơ chế trong câu hỏi khác đều đúng d) Many-to-one và one-to-one

Đáp án: A - One-to-one và many-to-many tránh hiện tượng blocking toàn bộ thread vì có nhiều kernel thread hỗ trợ song song. Many-to-one thì không.

94. [HK211] Với cơ chế *standard swapping*, để kích hoạt việc swap một process ra khỏi bộ nhớ, hệ điều hành thường dựa trên sự chuyển trạng thái nào?

a) Ready → Running b) Running → Ready
c) Running → Terminated d) Running → Waiting

Đáp án: D - Khi một tiến trình đang thực thi (Running) phát ra một yêu cầu nhập/xuất (I/O request) hoặc chờ một sự kiện nào đó, nó sẽ chuyển sang trạng thái chờ (Waiting hay Blocked). Các thao tác I/O (như đọc đĩa, chờ người dùng gõ phím) thường tốn một khoảng thời gian rất dài so với tốc độ của CPU.

95. [HK211] Cho hệ thống có 4 process như sau, sử dụng giải thuật *Shortest Remaining Time First (SRTF)*. Process đến trước được ưu tiên khi hòa.

Process	Arrival Time	Burst Time
P1	0	6
P2	1	3
P3	2	1
P4	3	2

Tại thời điểm 4 quá trình nào đang thực thi

a) P3 b) P1 c) P2 d) P4

Đáp án: C

96. [HK211] Cho các process sau:

```
Process P0
{
while(true)
{
    wait(s0);

    print("Hello");

    signal(s1);
    signal(s2);
}
}
```

```
Process P1
{
while(true)
{
    wait(s1);
    signal(s0);
}
}

Process P2
{
while(true)
{
    wait(s2);
    signal(s0);
}
}
```

Giả sử ban đầu:

$$s0 = 1, \quad s1 = 0, \quad s2 = 0$$

Số lần tối thiểu chuỗi "Hello" được in ra là:

a) Ít nhất 2 lần b) Ít nhất 3 lần c) Đúng 2 lần d) Đúng 3 lần

Đáp án: B - Bước 1: Lượt chạy đầu tiên của P0, Bước 2: P1 và P2 được đánh thức Kịch bản xấu nhất (Đồng thời / Binary Semaphore): Trong môi trường đại học, các bài toán dạng này thường ngầm định kiểm tra trường hợp xấu nhất với Semaphore nhị phân (Binary Semaphore - giá trị tối đa là 1). Nếu P1 và P2 được hệ điều hành cho chạy gần như đồng thời và cùng gọi signal(s0), giá trị của s0 lớn nhất cũng chỉ được set lên 1 (đồng nghĩa với việc 1 tín hiệu đã bị ghi đè/mất). Lúc này, P0 chỉ nhận được 1 token duy nhất để qua ải wait(s0). P0 tiếp tục chạy và in ra chuỗi "Hello" (lần thứ 2). Sau đó nó lại nhả s1 và s2 để tiếp tục chu kỳ.

97. [HK211] Tại sao các thread cần có thanh ghi (register) và stack riêng, trong khi code, data và files có thể chia sẻ?

a) Giúp hệ điều hành ghi nhớ thread đang chạy trên CPU/lõi nào
b) Vì các thread luôn chạy trên các CPU khác nhau
c) Vì các thread thực thi độc lập và register/stack chứa trạng thái thực thi riêng của từng thread
d) Giúp thread tăng tốc độ xử lý

Đáp án: C

98. [HK211] Chờ đợi có giới hạn (*bounded waiting*) nghĩa là tồn tại một giới hạn về số lần các quá trình khác được phép vào vùng tranh chấp:

a) Sau khi một quá trình yêu cầu vào vùng tranh chấp và trước khi yêu cầu đó được chấp nhận
b) Sau khi các quá trình khác đều hoàn tất n lần vùng tranh chấp
c) Trước khi một quá trình yêu cầu vào vùng tranh chấp
d) Khi một quá trình khác đang ở trong vùng tranh chấp

Đáp án: A - Bounded waiting yêu cầu: Request → Waiting → Critical Section phải tồn tại một giới hạn hữu hạn về số lần các process khác được vào critical section trước khi process đang chờ được phục vụ.

99. [HK211] Một semaphore có giá trị khởi tạo: $S = 5$ Các process lần lượt thực hiện: 9 lần *wait*() và: 3 lần *signal*() Sau khi hoàn tất, phát biểu nào đúng?

a) Không có process nào đang đợi trên semaphore
b) Có 5 process có thể đồng thời vào vùng tranh chấp

c) Nếu có yêu cầu mới đi vào vùng tranh chấp thì duy nhất 1 yêu cầu được đáp ứng
d) Có 1 process đang đợi trên semaphore

Đáp án: D - Vì $S = 5 - 9 + 3 = -1 < 0$ nên có $|S| = 1$ process đang bị block (đang đợi).

100. [HK211] Ngắt nào được sử dụng để phục vụ quá trình định thời (CPU scheduling) trong hệ điều hành?

a) Ngắt đồng bộ và ngắt I/O
b) Ngắt mềm (trap)
c) Ngắt đồng hồ (timer interrupt)
d) Ngắt I/O

Đáp án: C - Vì hệ điều hành dùng ngắt đồng hồ (timer interrupt) để tạo các tín hiệu theo chu kỳ thời gian, giúp thực hiện preemption (thu hồi CPU), chuyển ngữ cảnh (context switch) và hỗ trợ các thuật toán định thời như Round Robin.

101. [HK211] Cho một hệ thống gồm 4 process có thông tin như sau:

Process	Thời điểm đến	Burst time
P1	0	6
P2	1	3
P3	2	1
P4	3	2

Các process được cấp phát CPU theo giải thuật Shortest Remaining Time First (SRTF). Tính thời gian đợi trung bình.

a) 1,25
b) 2,75
c) 3,25
d) 2,25

Đáp án: D

102. [HK211] Cho một hệ thống gồm 5 process, tất cả đều đến tại thời điểm 0. Tính thời gian đợi trung bình khi sử dụng giải thuật Round Robin (RR) với quantum time $q = 1$.

Process	Burst time
P1	5
P2	3
P3	6
P4	2
P5	1

a) 9
b) 6
c) 7
d) 8

Đáp án: D

103. [HK211] Để định thời hệ thống cho các process thiên về tính toán (CPU-intensive process), giải thuật định thời CPU nào hợp lý nhất?

a) SJF
b) SRTF
c) Priority (non-preemptive)
d) Round Robin

Đáp án: A - CPU-intensive processes thường có CPU burst dài và ít I/O. SJF (Shortest Job First) là tối ưu về thời gian chờ trung bình vì luôn ưu tiên process có burst nhỏ hơn trước, giúp giảm tổng thời gian chờ và tăng hiệu quả sử dụng CPU trong hệ thống batch/CPU-bound.

104. [HK211] Chọn phát biểu đúng về chuyển ngữ cảnh đối với user-level thread:

a) Chuyển ngữ cảnh giữa hai thread không tốn CPU
b) Chuyển ngữ cảnh giữa hai thread do bộ định thời thực hiện
c) Chuyển ngữ cảnh giữa hai quá trình tốn ít chi phí CPU hơn giữa hai thread
d) Chuyển ngữ cảnh giữa hai thread tốn ít chi phí CPU hơn giữa hai quá trình

Đáp án: D - Vì user-level thread được quản lý ở user space nên chuyển ngữ cảnh giữa các thread nhẹ hơn nhiều so với process context switching (không cần chuyển không gian địa chỉ, không cần thay đổi PCB đầy đủ). Do đó chi phí CPU của thread context switch thấp hơn process context switch.

105. [HK211] Chọn câu đúng về so sánh multiprogramming và time-sharing (multitasking):

a) Multiprogramming làm giảm hiệu suất sử dụng CPU còn time-sharing thì làm tăng hiệu suất sử dụng CPU
b) Hệ thống multiprogramming không chú trọng khả năng tương tác hiệu quả với người dùng, còn time-sharing không cung cấp khả năng tương tác hiệu quả với người dùng
c) Trái với ý B
d) Multiprogramming giữ nhiều công việc trong bộ nhớ còn time-sharing chỉ giữ một

Đáp án: C - Trái với ý B

106. [HK211] Cho 3 công việc A, B, C có mức độ ưu tiên cao, trung bình, thấp. A và C có thao tác lên biến chia sẻ. A bị chặn gián tiếp bởi B vì C đang giữ tài nguyên trong vùng găng. Để giải quyết, hệ thống cho phép A tạm thời nhường mức ưu tiên cho C để C hoàn thành và giải phóng tài nguyên. Kỹ thuật này gọi là gì?

a) Đảo ngược độ ưu tiên (Priority Inversion)
b) Nâng cấp độ ưu tiên (Priority Boosting)
c) Chỉnh sửa độ ưu tiên
d) Loại bỏ độ ưu tiên

Đáp án: B - Kỹ thuật mô tả trong đề là priority inheritance (thừa kế độ ưu tiên), trong đó tiến trình giữ tài nguyên (C) được tạm thời nâng mức ưu tiên để tránh tình trạng priority inversion kéo dài.

107. [HK211] item Một thread bao gồm:

a) PC, stack, register và signal
b) PC, code
c) PC, code, stack
d) PC, stack, register

Đáp án: D - PC, stack, register

108. [HK181] Cho đoạn mã sử dụng semaphore sau:

```
Semaphore S1, S2;
S1.value = 0;
S2.value = 1;

Process P1:
while(1) {
    wait(S1);
    // Critical section
    signal(S2);
}

Process P2:
while(1) {
    wait(S2);
    // Critical section
    signal(S1);
}
```

Phát biểu nào sau đây đúng?

a) P1 và P2 đều không thể vào vùng tranh chấp.
b) P1 và P2 có thể đồng thời vào vùng tranh chấp.

c) P1 không thể vào, P2 có thể vào vùng tranh chấp.
d) P1 và P2 luân phiên vào vùng tranh chấp.

Đáp án: D - Ban đầu: S1 = 0, S2 = 1, P2 chạy trước vì S2 = 1 nên P2 vào critical section, Sau khi P2 xong: signal(S1) → S1 = 1, Lúc này P1 mới được vào, P1 xong: signal(S2) → S2 = 1, Hai tiến trình thay phiên nhau vào vùng tranh chấp (strict alternation)

Đáp án: D

109. [HK181] Trong thuật toán lập lịch *Round Robin (RR)*, khi một tiến trình sử dụng hết *quantum time*, nó sẽ chuyển sang trạng thái nào?

a) Chuyển từ trạng thái Running sang Ready.
b) Chuyển từ trạng thái Running sang Blocked.
c) Chuyển từ trạng thái Ready sang Running.
d) Chuyển từ trạng thái Running sang Terminated.

Đáp án: A - Trong Round Robin, khi hết quantum time, hệ điều hành sẽ thu hồi CPU của tiến trình đang chạy và đưa nó trở lại hàng đợi Ready để chờ lượt tiếp theo.

110. [HK181] Trạng thái của một tiến trình (process state) thể hiện điều gì?

a) Hoạt động vừa được thực hiện của tiến trình.
b) Hoạt động hiện tại của tiến trình.
c) Hoạt động sẽ được thực hiện tiếp theo của tiến trình.
d) Hoạt động cuối cùng của tiến trình.

Đáp án: B - Trạng thái tiến trình biểu diễn tình trạng hiện tại của tiến trình trong hệ điều hành (ví dụ: Ready, Running, Waiting, Terminated), tức là nó đang ở bước xử lý nào tại thời điểm hiện tại.

111. [HK181] Chọn phát biểu đúng về chuyển ngữ cảnh (context switching) đối với *user-level thread*:

a) Chuyển ngữ cảnh giữa hai thread không tốn chi phí sử dụng CPU.
b) Chuyển ngữ cảnh giữa hai thread ít tốn chi phí hơn chuyển ngữ cảnh giữa hai tiến trình.
c) Chuyển ngữ cảnh giữa hai thread do bộ định thời (scheduler) của hệ điều hành thực hiện.
d) Chuyển ngữ cảnh giữa hai tiến trình ít tốn chi phí hơn chuyển ngữ cảnh giữa hai thread.

Đáp án: B - User-level threads có chi phí chuyển ngữ cảnh thấp hơn process context switch vì không cần chuyển đổi không gian địa chỉ (address space), chỉ cần lưu/khôi phục trạng thái thread trong user space.

112. [HK181] Kỹ thuật nào sau đây **không phải** là một cơ chế dùng để tránh xung đột dữ liệu (race condition)?

a) Mutex lock
b) Barrier
c) Semaphore
d) Spinlock

Đáp án: B - Barrier chủ yếu dùng để đồng bộ hóa các luồng tại một điểm (synchronization point), không trực tiếp dùng để bảo vệ vùng găng (critical section) hay tránh race condition như mutex, semaphore, spinlock.

113. [HK181] Những giải thuật lập lịch CPU nào sau đây thuộc nhóm *Non-Preemptive scheduling*?

a) First Come First Serve (FCFS), Shortest Job First (SJF), Round Robin (RR)
b) SJF, SRTF, Priority Based (Preemptive), Round Robin
c) FCFS, SJF, Priority Based (Non-preemptive)
d) FCFS, SRTF, Round Robin

Đáp án: C - Non-preemptive scheduling gồm FCFS, SJF và Priority (non-preemptive). SRTF và Round Robin thuộc nhóm preemptive scheduling.

114. [HK181] Trong các chiến lược định thời nào mà một process đang thực thi không thể bị chiếm quyền thực thi bởi process khác?

a) FCFS và Shortest Remaining Time First
b) Shortest Job First và FCFS
c) Shortest Remaining Time First và Round Robin
d) Round Robin và Shortest Job First

Đáp án: B - FCFS và SJF đều là Non-preemptive scheduling, nghĩa là tiến trình khi đã được cấp CPU sẽ chạy đến khi kết thúc hoặc tự chờ I/O, không bị tiến trình khác chiếm CPU. SRTF và Round Robin đều là Preemptive.

115. [HK181] Để định thời cho một hệ thống có các quá trình đa dạng (phân bố đều giữa CPU-bound và I/O-bound), giải pháp nào sau đây là hợp lí nhất?

a) Shortest Remaining Time First
b) Multilevel Queue
c) Shortest Job First
d) Multilevel Feedback Queue

Đáp án: D - Multilevel Feedback Queue là giải thuật linh hoạt nhất, tự động điều chỉnh mức ưu tiên của tiến trình dựa trên hành vi CPU/I-O, phù hợp với hệ thống có workload đa dạng.

116. [HK181] Lệnh nào sau đây dùng để lấy Process ID của Process cha (parent process) của tiến trình hiện tại?

a) getpid()
b) getepid()
c) getppid()
d) getfpid()

Đáp án: C - getppid() trả về Process ID của tiến trình cha. Trong khi đó getpid() trả về Process ID của chính tiến trình hiện tại.

117. [HK181] Chọn phát biểu đúng về *process* và *thread*:

a) Các thread có thể đọc, ghi trong cùng không gian bộ nhớ còn các process thì không.
b) Các thread có chung vùng stack còn các process thì có vùng stack riêng biệt.
c) Các process có thể giao tiếp với nhau còn các thread thì không.
d) Cả A, B và C đều sai.

Đáp án: A - Các thread trong cùng một process chia sẻ chung không gian địa chỉ (memory space), nên có thể đọc/ghi dữ liệu chung. Ngược lại, các process có không gian bộ nhớ tách biệt.

118. [HK181] Quá trình P1 thực hiện lệnh $X = X + 1$ và P2 thực hiện lệnh $X = X - 1$, trong đó X là biến chia sẻ. Đây là ví dụ của:

a) Race condition
b) Producer - Consumer problem
c) Bounded buffer problem
d) Cả 3 đều đúng

Đáp án: A - Đây là ví dụ điển hình của race condition, và hai tiến trình cùng truy cập và cập nhật biến chia sẻ X mà không có cơ chế đồng bộ, dẫn đến kết quả không xác định.

119. [HK181] Sử dụng thuật toán định thời CPU *Shortest Job First (SJF - Non-preemptive)*, thứ tự thực thi các process trong bảng sau là:

Process	Arrival Time	Burst Time
P0	0	10
P1	3	6
P2	5	3
P3	7	3

a) P0, P2, P3, P1
b) P0, P1, P2, P3
c) P1, P0, P2, P3
d) P2, P3, P1, P0

Đáp án: A

120. [HK181] Chọn phát biểu **sai** về các giải thuật định thời CPU:

a) Giải thuật Shortest Remaining Time First (SRTF) có chế độ quyết định là Non-preemptive.
b) Giải thuật First Come First Serve (FCFS) không bao giờ gây ra starvation.
c) Muốn xây dựng giải thuật định thời tối ưu hoàn toàn cần phải biết trước tương lai.
d) Giải thuật Round Robin (RR) nếu có quantum time nhỏ sẽ tăng tính đáp ứng hệ thống.

Đáp án: A - SRTF là thuật toán Preemptive (có thể tước quyền CPU của tiến trình đang chạy), không phải Non-preemptive.

2 câu tiếp theo Cho các tiến trình P1, P2, P3, P4, P5 đều đến tại thời điểm 0 với bảng sau:

Process	Burst Time
P1	10
P2	1
P3	2
P4	4
P5	5

121. [HK181] Tính *Average Turnaround Time* khi sử dụng Round Robin (quantum = 2).

a) 4.4 b) 8.8 c) 12.2 d) 6.0

Đáp án: C - 12.2

122. [HK181] Tính *Average Waiting Time* khi sử dụng SJF (non-preemptive).

a) 6.2 b) 5.2 c) 4.6 d) 4.2

Đáp án: C - 4.6

123. [HK181] Một thread bao gồm các thông tin nào sau đây?

a) Program counter, Code.
b) Program counter, Code, Stack.
c) Program counter, Stack và các Register.
d) Program counter, Stack, Register và Signal.

Đáp án: C - Mỗi thread có riêng Program Counter (PC), Stack và tập thanh ghi (register set). Trong khi đó code và data thường được chia sẻ giữa các thread trong cùng process.

124. [HK181] Các user-thread trong cùng một process chia sẻ các thông tin nào sau đây?

a) Phần code và Program Counter của chương trình.
b) Các quá trình con của chương trình.
c) Code và data của process.
d) Cả code, data và Program Counter của chương trình.

Đáp án: C - Các thread trong cùng process chia sẻ code section, data section và heap. Mỗi thread vẫn có PC và stack riêng.

125. [HK181] Định thời ngắn hạn (*short-term scheduler*) có nhiệm vụ gì?

a) Chọn process nào sẽ đưa vào hàng đợi ready.
b) Chọn process nào sẽ được cấp CPU để thực thi kế tiếp.
c) Chọn process nào sẽ bị swap ra bộ nhớ phụ.
d) Chọn chương trình nào sẽ được nạp vào bộ nhớ.

Đáp án: B - Short-term scheduler (CPU scheduler) quyết định tiến trình nào trong hàng đợi ready sẽ được cấp CPU tiếp theo.

126. [HK181] Giả sử một process đang ở trạng thái *Blocked* do chờ thao tác I/O. Khi yêu cầu I/O hoàn thành, process sẽ chuyển sang trạng thái nào?

a) Running b) Suspended c) Terminated d) Ready

Đáp án: D - Khi thao tác I/O hoàn tất, process không chạy ngay mà được đưa trở lại hàng đợi Ready để chờ CPU cấp phát.

127. [HK181] Khối chức năng nào chịu trách nhiệm chuyển CPU cho process được chọn bởi *short-term scheduler*?

a) Scheduler
b) Dispatcher
c) Interrupt
d) Cả A, B, C đều sai

Đáp án: B - Dispatcher là thành phần thực hiện context switch, chuyển CPU sang process đã được short-term scheduler lựa chọn.

128. [HK181] Cho đoạn chương trình sau:

```
int main(int argc, char * argv[])
{
    int pid = fork();

    if (pid == 0)
    {
        fork();
        fork();
        printf("Chuc ban lam bai tot\n");
    }
    else
    {
        fork();
    }
}
```

Có bao nhiêu dòng "Chuc ban lam bai tot" được in ra?

a) 2 b) 3 c) 4 d) 8

Đáp án: C - 2 fork trong if sẽ nhân đôi lên 4

129. [HK181] Quá trình (*process*) là gì?

a) Một chương trình có thể thực thi được.
b) Một thủ tục/hàm trong chương trình.
c) Một chương trình đang thực thi.
d) Một tập tin nhị phân có phần mở rộng .doc hoặc .pdf.

Đáp án: C - Process là một chương trình đang được thực thi trong bộ nhớ, bao gồm code, dữ liệu, trạng thái CPU và các tài nguyên liên quan.

130. [HK181] Cho đoạn chương trình sau:

```
int main(int argc, char *argv[]){
    int pid = fork();

    if (pid == 0) {
        fork();
        fork();
        fork();
    }
    else {
        fork();
        fork();
    }
}
```

Hỏi có bao nhiêu process được tạo ra?

a) 6 b) 8 c) 10 d) 12

*Đáp án: D - if chia làm 2 nhánh 2*2*2 và 2*2*

131. [HK181] Hệ thống có 4 process như sau, sử dụng thuật toán *SRTF (Shortest Remaining Time First)*. Nếu nhiều process có cùng remaining time thì process đến trước được ưu tiên.

Process	Arrival Time	CPU Burst
A	0	8
B	2	4
C	3	6
D	6	4

Hãy tính **Average Waiting Time**.

a) 5.15 b) 5.25 c) 5.50 d) 5.75

Đáp án: B - 5.25

132. [HK181] Một thread bị block sẽ làm toàn bộ các thread khác trong cùng process bị block nếu hệ thống sử dụng mô hình multithreading nào?

a) One-to-One
b) Many-to-One
c) Many-to-Many
d) Tất cả đều sai

Đáp án: B - Trong mô hình Many-to-One, nhiều user threads được ánh xạ vào một kernel thread. Khi một thread bị block, toàn bộ process bị block theo.

133. [HK181] Trong hệ thống có nhiều CPU, các process được gọi là thực thi song song thực sự khi nào?

a) Cùng chia sẻ thời gian của một CPU.
b) Được thực thi trên các CPU khác nhau tại cùng một thời điểm.
c) Cùng truy cập dữ liệu dùng chung.
d) Có tương tác hoặc giao tiếp với nhau.

Đáp án: B - Parallelism xảy ra khi nhiều process/thread thực sự chạy đồng thời trên các CPU hoặc các lõi khác nhau cùng lúc.

134. [HK181] Khi nhiều process thực thi đồng thời trên cùng dữ liệu và kết quả cuối cùng phụ thuộc vào thứ tự thực thi của các process, hiện tượng này được gọi là gì?

a) Dynamic condition
b) Race condition
c) Essential condition
d) Critical condition

Đáp án: B - Race condition xảy ra khi nhiều process/thread cùng truy cập dữ liệu chia sẻ mà không đồng bộ đúng cách, làm kết quả phụ thuộc vào thứ tự thực thi.

135. [HK181] Khi một process đang ở trong vùng tranh chấp (*critical section*), không process nào khác được phép vào vùng đó. Điều kiện này gọi là gì?

a) Bounded waiting
b) Progress
c) Mutual exclusion
d) Critical exclusion

Đáp án: C - Mutual exclusion đảm bảo tại một thời điểm chỉ có một process/thread được thực thi trong critical section.

136. [HK181] Chọn phát biểu đúng về *critical section* (vùng tranh chấp):

a) Là đoạn mã lệnh được chia sẻ giữa nhiều process hoặc thread.
b) Là vùng dữ liệu được chia sẻ giữa nhiều process hoặc thread.
c) Là vùng dữ liệu được chia sẻ giữa nhiều process hoặc thread thực thi đồng thời.
d) Là đoạn mã truy xuất vào vùng dữ liệu chia sẻ.

Đáp án: D - Critical section là đoạn mã mà trong đó process/thread truy cập tài nguyên hoặc dữ liệu dùng chung, cần cơ chế đồng bộ để tránh race condition.

137. [HK181] Để đồng bộ các thread trong cùng một process, có thể sử dụng cơ chế nào sau đây?

a) Semaphore b) Monitor c) Bakery algorithm d) Cả 3 đều đúng

Đáp án: D - Semaphore, monitor và bakery algorithm đều là các cơ chế/hướng tiếp cận dùng để đồng bộ thread và giải quyết bài toán critical section.

138. [HK181] Chọn phát biểu đúng khi so sánh các kỹ thuật đồng bộ:

a) Với kỹ thuật busy-waiting, process đang chờ sẽ chuyển sang trạng thái blocked.
b) Với kỹ thuật không busy-waiting, khi process được wake-up sẽ lập tức chuyển sang running.
c) Busy-waiting giúp giảm số lần context switch so với kỹ thuật không busy-waiting.
d) Kỹ thuật không busy-waiting sử dụng CPU kém hiệu quả hơn busy-waiting.

Đáp án: C - Busy-waiting khiến process liên tục kiểm tra điều kiện mà không bị block, nên giảm context switching nhưng lại tốn CPU hơn.

139. [HK181] Cơ chế loại trừ tranh chấp nào sau đây **không** sử dụng busy-waiting?

a) Peterson algorithm b) TestAndSet c) Semaphore d) Dekker algorithm

Đáp án: C - Semaphore (blocking semaphore) cho phép process chuyển sang trạng thái waiting/block thay vì quay vòng liên tục như các kỹ thuật busy-waiting.

140. [HK181] Cơ chế đồng bộ nào sau đây có hiệu quả sử dụng CPU tốt nhất?

a) Peterson algorithm b) TestAndSet c) Dekker algorithm d) Semaphore

Đáp án: D - Semaphore không busy-waiting giúp process bị chặn khi chờ tài nguyên, tránh lãng phí CPU nên hiệu quả hơn các cơ chế spin-waiting.

141. [HK181] Chiến lược định thời nào cho phép process đang chạy có thể bị tạm ngưng để nhường CPU cho process khác?

a) Shortest Remaining Time First b) Non-preemptive scheduling
c) Shortest Job First d) FCFS

Đáp án: A - SRTF là giải thuật preemptive, cho phép hệ điều hành tước CPU khỏi process đang chạy nếu có process khác với remaining time nhỏ hơn.

142. [HK181] Các process có thời điểm đến và CPU burst time như sau. Sử dụng giải thuật *SRTF (Shortest Remaining Time First)*. Hãy tính **Average Waiting Time**.

Process	Arrival Time	CPU Burst Time
P1	0	6
P2	1	2
P3	3	4
P4	4	1

a) 3.25 b) 2.0 c) 2.25 d) 3.0

Đáp án: B - 2.0

143. [HK181] Giải thuật định thời CPU nào phù hợp nhất cho các ứng dụng tương tác?

a) Shortest Remaining Time First b) Round Robin
c) Shortest Job First d) FCFS

Đáp án: B - Round Robin chia CPU theo quantum time giúp các process được phản hồi nhanh và công bằng, phù hợp với hệ thống tương tác.

144. [HK161] Chọn phát biểu đúng:

a) Các thread trong cùng một process không thể truy cập dữ liệu toàn cục của process.
b) Trên hệ thống multiprocessor, các thread của một process dùng mô hình many-to-one không thể thực thi song song.
c) Trên hệ thống single processor, các process có thể thực thi song song nếu dùng các chỉ thị thích hợp.
d) Trên hệ thống single processor, các thread của một process dùng mô hình one-to-one có thể thực thi song song.

Đáp án: B - Trong mô hình many-to-one, nhiều user threads ánh xạ vào một kernel thread duy nhất, nên dù hệ thống có nhiều CPU thì các thread cũng không thể chạy song song thực sự.

145. [HK161] Chuyển ngữ cảnh (*context switch*) là quá trình nào sau đây?

a) Process thực thi xong một câu lệnh và nạp câu lệnh mới để tiếp tục thực thi.
b) Tạm dừng process và chuyển nó ra đĩa cứng.
c) Hệ điều hành lưu trạng thái của process hiện tại và nạp trạng thái của process khác để tiếp tục thực thi.
d) Process chuyển từ trạng thái New sang Ready.

Đáp án: C - Context switch xảy ra khi CPU chuyển từ process này sang process khác. Hệ điều hành phải lưu trạng thái của process cũ vào PCB và khôi phục trạng thái của process mới.

146. [HK161] Định thời ngắn hạn (*short-term scheduler*) là gì?

a) Chọn process nào sẽ được đưa vào hàng đợi ready.
b) Chọn process nào sẽ được cấp CPU để thực thi tiếp theo.
c) Chọn process nào sẽ được swap ra bộ nhớ phụ.
d) Chọn chương trình nào sẽ được nạp vào bộ nhớ.

Đáp án: B - Short-term scheduler (CPU scheduler) quyết định process nào trong hàng đợi ready sẽ được cấp CPU để chạy tiếp theo.

147. [HK161] Giả sử một process đang ở trạng thái *Blocked* do yêu cầu thao tác I/O. Khi thao tác I/O hoàn thành, process sẽ chuyển sang trạng thái nào?

a) Running b) Terminated c) Suspended d) Ready

Đáp án: D - Sau khi thao tác I/O hoàn tất, process được đưa trở lại hàng đợi Ready để chờ CPU cấp phát, chứ không chạy ngay lập tức.

148. [HK161] Khi một process đang thực thi trong vùng tranh chấp (*critical section*), không process nào khác được phép thực thi trong vùng đó. Điều kiện này được gọi là gì?

a) Mutual exclusion
b) Critical exclusion
c) Synchronous exclusion
d) Asynchronous exclusion

Đáp án: A - Mutual exclusion đảm bảo tại một thời điểm chỉ có một process hoặc thread được truy cập vùng tranh chấp để tránh race condition.

149. [HK161] Khi một process sử dụng hết *quantum time* được cấp phát trong thuật toán Round Robin, process sẽ:

a) Chuyển từ trạng thái Running sang Ready.
b) Chuyển từ trạng thái Running sang Blocked.
c) Chuyển từ trạng thái Running sang Suspended-Ready.
d) Chuyển từ trạng thái Running sang Suspended.

Đáp án: A - Khi hết quantum time, hệ điều hành thu hồi CPU và đưa process trở lại hàng đợi Ready để chờ lượt thực thi tiếp theo.

150. [HK161] Trạng thái của một process thể hiện điều gì?

a) Hoạt động vừa được thực hiện của process.
b) Hoạt động hiện tại của process.
c) Hoạt động sẽ được thực hiện tiếp theo của process.
d) Hoạt động cuối cùng của process.

Đáp án: B - Process state biểu diễn tình trạng hiện tại của process trong hệ điều hành, ví dụ: Ready, Running, Waiting, Terminated.

151. [HK161] Quá trình (*process*) là gì?

a) Một chương trình có thể thực thi được.
b) Một thủ tục với chức năng xác định.
c) Một chương trình được nạp vào bộ nhớ để thực thi.
d) Một tập tin có phần mở rộng .bin, .exe hoặc .sh.

Đáp án: C - Process là một chương trình đang được thực thi trong bộ nhớ, bao gồm code, dữ liệu, trạng thái CPU và các tài nguyên liên quan.

152. [HK161] Chọn phát biểu đúng về chuyển ngữ cảnh đối với *user-level thread*:

a) Chuyển ngữ cảnh giữa hai thread không tốn chi phí sử dụng CPU.
b) Chuyển ngữ cảnh giữa hai thread ít tốn chi phí hơn chuyển ngữ cảnh giữa hai process.
c) Chuyển ngữ cảnh giữa hai thread do bộ định thời (scheduler) của hệ điều hành thực hiện.
d) Chuyển ngữ cảnh giữa hai process ít tốn chi phí hơn chuyển ngữ cảnh giữa hai thread.

Đáp án: B - Chuyển ngữ cảnh giữa các thread (đặc biệt user-level thread) nhẹ hơn so với process vì không cần chuyển đổi toàn bộ không gian địa chỉ (address space), chỉ thay đổi trạng thái thread.

153. [HK161] Trong các chuỗi chuyển đổi trạng thái của một process, chuỗi nào là hợp lệ?

a) New → Ready → Running → Ready → Terminated
b) Ready → Running → Terminated
c) New → Ready → Terminated
d) New → Ready → Running → Terminated

Đáp án: D - Chuỗi trạng thái chuẩn của process là New → Ready → Running → Terminated. Process có thể bị chuyển từ Running về Ready nếu bị preempt.

154. [HK161] Trong giải thuật định thời Round Robin, khi giảm nhát cắt thời gian (quantum time):

a) Tăng sự công bằng đối với process dạng I/O-bound.
b) Phí tổn chuyển ngữ cảnh (context switch overhead) sẽ giảm.
c) Độ lợi CPU (CPU utilization) sẽ tăng.
d) Thời gian đáp ứng (response time) sẽ tăng.

Đáp án: A - Khi quantum nhỏ hơn, các process I/O-bound được phản hồi nhanh hơn, cải thiện tính công bằng và khả năng tương tác.

155. [HK161] Trên cùng một hệ điều hành, các thread trong mô hình nào sau đây có khả năng được gán nhiều CPU nhất?

a) One-to-one
b) Many-to-many
c) Many-to-one
d) Cả 3 câu trên đều sai

Đáp án: B - Many-to-many cho phép ánh xạ nhiều user threads lên nhiều kernel threads, giúp tận dụng nhiều CPU đồng thời hiệu quả nhất.

Dữ kiện sau dây dùng cho 3 câu tiếp sau dây: Hệ thống có 4 process với thời điểm đến và CPU burst như sau:

Process	Arrival Time	CPU Burst
A	0	8
B	2	4
C	3	6
D	6	4

156. [HK161] Sử dụng FCFS, thời gian đợi trung bình là:

a) 5.25
b) 6.25
c) 6.75
d) Khác

Đáp án: C

157. [HK161] SJF (non-preemptive), tính waiting time trung bình:

a) 6.25
b) 6.75
c) 5.25
d) Khác

Đáp án: A

158. [HK161] Round Robin (q = 2), số lần lập lịch (context dispatch) từ t=0 đến t=22:

a) 11 b) 12 c) 13 d) Khác

Đáp án: A